

南開大學

《汇编语言与逆向技术》课程实验报告

实验三：Bubble



学 院 网络空间安全学院
专 业 物联网工程
学 号 2310422
姓 名 谢小珂
班 级 0410

一、实验目的

- 1、熟悉汇编语言的整数数组；
- 2、熟悉基址变址操作数、相对基址变址操作数；
- 3、掌握排序算法的底层实现细节

二、实验原理

1. 编写思路

本程序完全按照实验说明的流程来写，整体分成四个阶段：

（1）输入读取：先用 StdOut 给出提示，再用 StdIn 一次性读入一行最多 10 个、用空格或逗号分隔的十进制数，这样后面可以在内存里逐字节处理。

（2）字符串解析为整数数组：用 ESI 逐字符扫描输入缓冲区，遇到数字就做“原值 $\times 10$ +当前位”生成真正的整数，遇到空格或逗号就把累积的数写进 arr[i]。为适应实验里提到的“空格或逗号分隔”的不规范输入，增加了一个标志位，只在确实读到数字时才入数组，避免连续分隔符被当成 0。

（3）冒泡排序：按照 PPT 里的冒泡算法，用两层循环对数组中 10 个 DWORD 做相邻比较并交换，内层用基址变址方式访问 [arr + index*4]，体现本实验对数组寻址方式的要求。

（4）结果输出：排序完成后，逐个把数组元素取出，用 masm32 提供的 dwtoa 转成十进制字符串，再用 StdOut 打印并以空格分隔；最后加一次读取防止闪退。编译、链接则按 docx 给出的命令分别用 `ml /c /coff` 和 `link /SUBSYSTEM:CONSOLE` 生成控制台程序。

2. 汇编代码

```
001 ; -----
002 ; 功 能：从键盘一次性读入一行，行内包含 10 个无符号十进制整数，
003 ;         分隔符可以是空格也可以是逗号，甚至空格和逗号混着来。
004 ;         程序把这 10 个数解析出来后，存入数组中，用冒泡排序
005 ;         按从小到大排序，然后输出排序后的结果。
```

```

006 ; 环 境: MASM32, m1 6.14, 使用 masm32 库中的 StdIn / StdOut / dwtoa
007 ; 说 明: 加了一个 digitFlag 标志, 专门用来防止“连续分隔符
008 ;         被当成一个 0 存进数组”的错误; 最后还加了一个“按回车
009 ;         退出”, 防止双击运行时闪退。
010 ;-----
011 .386
012 .model flat, stdcall
013 option casemap:none           ; 大小写敏感, 避免符号被自动变成大写
014 ; 1) 头文件和库
015 ; windows.inc      基础的 Windows 常量/结构
016 ; kernel32.inc     ExitProcess 等基础 API
017 ; masm32.inc       StdIn/StdOut/dwtoa 这些简化过的 I/O 函数
018 include \masm32\include\windows.inc
019 include \masm32\include\kernel32.inc
020 include \masm32\include\masm32.inc
021
022 includelib \masm32\lib\kernel32.lib
023 includelib \masm32\lib\masm32.lib
024
025 ; 声明要用的 API (虽然 masm32.inc 里已经有, 但显示写更清楚)
026 ExitProcess PROTO :DWORD
027 ;-----
028 ; 2) 数据段
029 ;-----
030 .data
031 ; 给用户看的提示语: 告诉他要输入 10 个数, 用空格或逗号隔开
032 prompt      db "Enter 10 unsigned integers separated by space or comma:",13,10,0
033
034 ; 输入缓冲区: 一次性把用户那一行读进来
035 ; 120 字节足够放下 10 个数、分隔符和回车
036 inBuf       db 120 dup(0)
037
038 ; 真正存放 10 个整数的数组: 每个是 DWORD (dd), 一共 10 个
039 arr         dd 10 dup(0)
040
041 ; 输出时临时存放“整数转成的十进制字符串”的缓冲区
042 outBuf      db 12 dup(0)      ; 够放“4294967295”这种长度 +0
043
044 ; 输出时用的空格
045 spaceStr    db " ",0
046
047 ; 换行
048 crlf       db 13,10,0
049

```

```

050 ; 标志位:
051 ; digitFlag = 1 表示“刚刚读过至少一位数字，可以安全入数组”
052 ; digitFlag = 0 表示“目前只是分隔符，还不能入数组”
053 digitFlag db 0
054 ; 程序结束前的提示，防止闪退
055 exitMsg db "Press Enter to exit...",13,10,0
056 ;-----
057 ; 3) 代码段开始
058 ;-----
059 .code
060 main PROC
061     ; (1) 输出提示语
062     invoke StdOut, addr prompt
063
064     ; (2) 从键盘读入一行，最多 120 字节，放到 inBuf 里
065     ; StdIn 会以 0 结尾，所以后面可以用 0 判断行尾
066     invoke StdIn, addr inBuf, 120
067     ; (3) 初始化几个关键寄存器/标志
068     ; ESI 用来指向当前要处理的那个字符
069     ; EBX 用来累积当前正在读的这个整数（十进制转整数）
070     ; EDI 用来表示当前读到第几个数了（作为数组下标 0..9）
071     ;-----
072     mov esi, OFFSET inBuf ; ESI 指向输入缓冲区开头
073     xor ebx, ebx ; EBX = 0，准备开始累积第一个数
074     xor edi, edi ; EDI = 0，当前还没存任何数
075     mov byte ptr digitFlag, 0 ; 暂时还没读到数字
076 ;=====
077 ; 解析输入这一大段：逐字符扫描，根据字符类型做不同处理
078 ;=====
079 parse_loop:
080     mov al, [esi] ; AL = 当前字符
081     ;-----
082     ; 遇到字符串结尾 or CR or LF
083     ; 说明这一行到头了，要把最后一个数入数组
084     ;-----
085     cmp al, 0
086     je store_last
087     cmp al, 13
088     je store_last
089     cmp al, 10
090     je store_last
091     ;-----
092     ; 判断是不是分隔符（空格或逗号）
093     ; 如果是分隔符，就看刚才有没有读过数字

```

```

094 ;-----
095 cmp     al, ' '
096 je      maybe_store
097 cmp     al, ','
098 je      maybe_store
099 ;=====
100 ; 能走到这里说明这个字符一定是 '0'~'9' 之间的数字
101 ; 我们要做的就是：把 EBX 里原来的值 *10，再加上这一位
102 ; 十进制乘 10：这里用了移位+加法的写法，避免调用 Mul
103 ;=====
104 sub     al, '0'                ; AL = 当前数字(0..9)
105 movzx   edx, al                ; EDX = 当前数字
106
107 mov     eax, ebx               ; EAX = 旧的数值
108 mov     ecx, eax               ; ECX = 旧的数值（要算 *2）
109
110 shl     eax, 3                 ; EAX = old * 8
111 shl     ecx, 1                 ; ECX = old * 2
112 add     eax, ecx               ; EAX = old * 10
113 add     eax, edx               ; EAX = old * 10 + 当前 digit
114
115 mov     ebx, eax               ; EBX = 新的数值
116 mov     byte ptr digitFlag, 1 ; 记下来：确实读到过数字了
117
118 jmp     next_char              ; 继续看下一个字符
119 ;-----
120 ; 遇到的是分隔符（空格/逗号）就会跳到这里
121 ; 我们要做的事是：只有在“刚刚确实读过数字”的情况下才入数组
122 ; 连续两个分隔符要直接跳过，不能存 0
123 ;-----
124 maybe_store:
125 cmp     byte ptr digitFlag, 1
126 jne     skip_store             ; 如果前面没读数字，说明是连续分隔符，别存
127
128 ;----- 真正存数的动作 -----
129 mov     [arr + edi*4], ebx      ; arr[edi] = 当前累积的这个数
130 inc     edi                    ; 下标 +1，准备存下一个数
131 xor     ebx, ebx               ; 当前数清零，重新累积
132 mov     byte ptr digitFlag, 0 ; 标志清零，下一轮要重新判断
133
134 ; 如果已经存了 10 个数，就没必要再解析了，直接去排序
135 cmp     edi, 10
136 jae     start_sort
137

```

```

138 ; 走到这里说明这次只是遇到了分隔符（或者连续分隔符）
139 ; 我们继续往后看下一个字符
140 skip_store:
141 next_char:
142     inc     esi                ; 指向输入缓冲区的下一个字符
143     jmp     parse_loop        ; 回去再判断一遍，是不是数字/分隔符/行尾
144 ;-----
145 ; 行尾处理:
146 ; 到了 '\0' / CR / LF 时，如果“最后一次是数字”，就要把这个数也存进去
147 ;-----
148 store_last:
149     cmp     byte ptr digitFlag, 1
150     jne     start_sort        ; 如果最后不是数字结尾，就不用存
151     mov     [arr + edi*4], ebx ; 存最后一个数
152     inc     edi                ; 数量 +1
153 ;=====
154 ; 排序阶段: 对 arr[0..9] 做标准冒泡排序（升序）
155 ; 外层 9 趟，内层每趟比较相邻元素，必要时交换
156 ;=====
157 start_sort:
158     mov     ecx, 9            ; 外层循环跑 9 次（10 个数需要 9 趟）
159
160 outer_loop:
161     mov     esi, OFFSET arr   ; 每一趟从 arr[0] 开始比
162     mov     edx, ecx          ; EDX 控制内层比较次数
163
164 inner_loop:
165     mov     eax, [esi]        ; EAX = 当前元素 arr[i]
166     cmp     eax, [esi+4]      ; 和 arr[i+1] 比较
167     jbe     no_swap           ; 如果 arr[i] <= arr[i+1] 则不交换
168
169     ; 需要交换: 把大的放到后面
170     xchg    eax, [esi+4]      ; 交换 EAX 和 arr[i+1]
171     mov     [esi], eax        ; 再把原来 arr[i+1] 写回前一个位置
172
173 no_swap:
174     add     esi, 4            ; ESI 指向下一个元素（DWORD 步长=4）
175     dec     edx
176     jnz     inner_loop        ; 这一趟还没比完就继续
177
178     dec     ecx
179     jnz     outer_loop        ; 外层还没做 9 趟就继续
180
181 ;=====

```

```

182 ; 输出阶段：把排好序的 10 个数一个个转成十进制字符串，再用 StdOut 打印出来
183 ;=====
184     mov     edi, 0                ; EDI 重新当输出时的下标 0..9
185
186 print_loop:
187     cmp     edi, 10
188     jae     done_print            ; 打完 10 个就结束
189
190     mov     eax, [arr + edi*4]    ; 取出第 edi 个数
191     ; dwtoa: dword to ascii, 把 EAX 里的数转成十进制字符串
192     invoke  dwtoa, eax, addr outBuf
193     invoke  StdOut, addr outBuf   ; 打印这个字符串
194
195     ; 除了最后一个，其余都输出一个空格
196     cmp     edi, 9
197     je      skip_space
198     invoke  StdOut, addr spaceStr
199
200 skip_space:
201     inc     edi
202     jmp     print_loop
203 ;-----
204 ; 输出结束：换行，然后提示用户按回车退出
205 ;-----
206 done_print:
207     invoke  StdOut, addr crlf     ; 换一行
208     invoke  StdOut, addr exitMsg  ; 打个“Press Enter...”防闪退
209     invoke  StdIn, addr inBuf, 120 ; 再读一次输入，这样窗口就不会秒关
210     ; 正常退出
211     invoke  ExitProcess, 0
212 main ENDP
213
214 END main

```

三、实验过程

1. 编译

使用 MASM32 提供的汇编器 ml 对源文件进行编译，将人可读的汇编指令转成机器可识别的目标文件。命令形式为：

```
\masm32\bin\ml /c /coff bubble_sort.asm
```

```
PS K:\大三上\汇编实验\实验报告\实验三> \masm32\bin\ml /c /coff bubble_sort.asm
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.
```

```
Assembling: bubble_sort.asm
```

```
*****
ASCII build
*****
```

2. 链接

编译完成后, 将生成的 bubble_sort.obj 与所需的运行库通过链接器 link 进行链接, 生成最终的可执行程序。命令形式为:

```
\masm32\bin\link /SUBSYSTEM:CONSOLE bubble_sort.obj
```

```
PS K:\大三上\汇编实验\实验报告\实验三> \masm32\bin\link /SUBSYSTEM:CONSOLE bubble_sort.obj
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.
```

3. 测试结果:

本实验选取了多组具有代表性的输入数据进行测试, 覆盖了常见输入、混合分隔符输入、最坏情况输入以及含有多余空白字符的输入等典型场景。

```
PS K:\大三上\汇编实验\实验报告\实验三> .\bubble_sort.exe
Enter 10 unsigned integers separated by space or comma:
9999 0 500 500 7 8 9 10 11 12
0 7 8 9 10 11 12 500 500 9999
Press Enter to exit...

PS K:\大三上\汇编实验\实验报告\实验三> .\bubble_sort.exe
Enter 10 unsigned integers separated by space or comma:
100, 3 50,7,9 0, 10000, 4 1 2
0 1 2 3 4 7 9 50 100 10000
Press Enter to exit...

PS K:\大三上\汇编实验\实验报告\实验三> .\bubble_sort.exe
Enter 10 unsigned integers separated by space or comma:
9,8,7,6,5,4,3,2,1,0
0 1 2 3 4 5 6 7 8 9
Press Enter to exit...

PS K:\大三上\汇编实验\实验报告\实验三> .\bubble_sort.exe
Enter 10 unsigned integers separated by space or comma:
20 5 17,3, 9 0 100 1 2 98
0 1 2 3 5 9 17 20 98 100
Press Enter to exit...
```

测试用例 1: 大数 + 重复值 + 0 混合

输入: 9999 0 500 500 7 8 9 10 11 12

输出: 0 7 8 9 10 11 12 500 500 9999

分析: 相比只测“简单递增/递减”的情况, 这一用例更能说明程序在“值域跨度大 + 存在重复”时仍然稳定。

测试用例 2: 空格与逗号混合的复杂输入

输入：100, 3 50,7 ,9 0, 10000, 4 1 2

输出：0 1 2 3 4 7 9 50 100 10000

分析：通过设置“读到数字才入数组”的标志位，程序成功过滤了多余分隔符，保证混合分隔输入也能还原出正确的 10 个数并排序。

测试用例 3：完全逆序输入（冒泡排序最坏情况）

输入：9,8,7,6,5,4,3,2,1,0

输出：0 1 2 3 4 5 6 7 8 9

分析：该用例验证了在冒泡排序的最坏情况下程序仍能正确升序输出，说明循环边界与交换逻辑都实现无误。

测试用例 4：开头含空白、分隔符混合、数值范围差异较大

输入：20 5 17,3, 9 0 100 1 2 98

输出：0 1 2 3 5 9 17 20 98 100

分析：该用例表明程序能忽略多余空白和混合分隔符，只保留真实输入的 10 个数并正确升序输出。

四、实验结论及心得体会

汇编语言数组操作知识点总结

1. 数组连续存储

程序将输入的 10 个整数先统一存入一段连续的内存中（如 `dd 10 dup(0)`），这样后续才能对同一批数据进行多次遍历、比较和交换。

2. 基址变址访问

访问数组元素时以数组首地址为基址，再用下标或指针作为偏移量，典型形式为 `[arr + index*4]` 或在排序中使用的 `[esi]`、`[esi+4]`。这种写法能直接定位到第 n 个元素，适合对相邻元素做比较。

3. 元素大小与步长一致

数组元素定义为 `DWORD`，占 4 字节，因此在循环中每次访问下一个元素都要加 4 字节（如 `add esi, 4`，或下标乘 4），保证一次只操作一个完整元素，防止地址错位。

4. 循环配合下标遍历

对数组的处理统一采用“初始化下标 → 进入循环 → 访问当前元素 → 下标/地址递增 → 判断结束”的模式。无论是把解析出的数依次写入数组，还是后面的冒泡排序，都是这种顺序遍历方式。

5. 先写入数组再统一处理

程序先把从键盘读到的字符解析成整数并存入数组，等数组中的 10 个元素都准备好之后，再对这片连续空间执行冒泡排序，最后一次性输出结果。这种“先数组化、后运算”的结构逻辑清晰，也便于调试和扩展。

心得体会

本次实验主要遇到几类问题：一是输入含有空格和逗号的混合分隔时，若连续出现分隔符容易被错误解析成 0，需要增加标志位判断是否真正读到数字；二是双击运行程序会立即退出，看不到排序结果，需要在结束前再读一次输入进行停顿；三是最初的字符串提示如果用中文且文件保存成 UTF-8，会在控制台出现乱码，需要改成英文或用 ANSI 保存；四是使用 MASM32 时不能直接用不存在的宏（如 CSTR），必须把提示语写在 .data 段再用 StdOut 输出。处理完这些问题后，输入解析、数组存储和排序才能稳定运行。